Livscykelhantering för 1177.se

Vi i grupp 5 kom överens om att ta en varsin tårtbit och jag ska här ta upp mina fördjupningar på 1177 ur ett livscykelperspektiv det är ju inte vilken webbplats som helst – det är en samhällskritisk e-tjänst som används av miljontals svenskar varje dag, som hanterar känsliga personuppgifter och som måste fungera dygnet runt, året om. 1177.se är mer som en plattform som knyter samman Sveriges alla regioner, deras journalsystem, vårdcentralernas bokningssystem och en mängd andra tjänster. Systemet förväntas att ha en lång livslängd och det utvecklas kontinuerligt med nya funktioner som digitala vårdmöten, egen provsvarshantering och påminnelser för vaccinationer.

När jag funderar på livscykelhantering i detta fall så insåg jag snabbt att detta inte handlar om att bygga något som är "färdigt". Det handlar om att skapa förutsättningar för att systemet ska kunna leva och utvecklas över tid, utan att kvaliteten eller stabiliteten offras.

Den icke-uppenbara utmaningen – 21 regioner med olika förutsättningar

Det som gör livscykelhantering extra komplex för 1177.se är inte tekniken i sig – det är styrningsmodellen. Systemet ägs och finansieras gemensamt av Sveriges 21 regioner, som alla har egna IT-miljöer, journalsystem och prioriteringar.

Vid första anblicken kan man tro att livscykelhantering "bara" handlar om att ha bra versionshantering och testning. Men för 1177.se handlar det lika mycket om:

- **Beslutsvägar:** En uppgradering som påverkar integrationen mot regionernas journalsystem kräver samordning med 21 olika IT-avdelningar. En region kan vara redo att uppgradera medan en annan inte är det.
- **Budgetcykler:** Regionernas budgetar beslutas årligen, vilket gör långsiktig planering svår. Det finns risk att medel för teknisk skuld prioriteras bort till förmån för akuta behov.
- **Kompetensförsörjning:** Systemet utvecklas ofta av konsulter med korta avtal. Detta gör att kunskap om systemets arkitektur och historiska designbeslut kan försvinna när konsulter byts ut.

Den insikten har varit viktig för mig: livscykelhantering för 1177.se handlar inte bara om teknik – det handlar minst lika mycket om organisation, avtal och samordning.

Riskbild – vad händer om vi försummar livscykelhantering?

När jag har funderat över risker har jag försökt att inte bara lista dem, utan att tänka i kedjor. En enskild risk är sällan farlig i sig – det är när flera risker samverkar som allvarliga incidenter uppstår.

Låt mig ta ett exempel som jag tror är högst troligt att det kan inträffa någon gång:

1. En region uppgraderar sitt journalsystem till en ny version
2. Ingen tydlig process finns för att testa integrationen mot 1177.se i förväg
3. Uppgraderingen görs under helgen när 1177.se:s testteam inte arbetar
4. På måndagsmorgon upptäcks att journaler från den regionen inte visas
5. Vårdpersonal kan inte se patienters journalhistorik
6. Patienter i den regionen kan inte få korrekt vård – vårdpersonal tvingas gissa eller skjuta upp behandling
7. Medierna rapporterar om patientsäkerhetsbrister
8. Förtroendet för hela 1177.se sjunker – även i regioner som inte drabbats

Det här är inget långsökt scenario. Det är en kedja som kan utlösas av en enda uppgradering i en enda region, om inte livscykelhanteringen är tillräckligt robust. Analyserar man riskerna och väger in sannolikhet att det inträffar och hur allvarlighetsgraden är för det.

Risk	Sannolikhet	Allvarlighetsgrad	Min reflektion
En region uppgraderar utan samordning	**Hög**	**Hög**	Regioner uppgraderar oberoende av varandra. Det är inte *om* det händer, utan *när*.
Integrationen testas inte i förväg	**Medel**	**Kritisk**	Finns testmiljöer som speglar produktion? Ofta inte – då blir första testet i produktion.
Patientsäkerheten påverkas	**Låg**	**Kritisk**	Sannolikheten är låg att någon faktiskt skadas, men *om* det sker är konsekvenserna extremt allvarliga.

Min slutsats är att risken för integrationsproblem är hög, och att vi måste hantera den som en kritisk risk – även om sannolikheten för patientsäkerhetspåverkan är låg. Det handlar om att bygga system som kan upptäcka och åtgärda problem snabbt när de ändå uppstår.

Teststrategi – hur verifierar vi livscykelhantering?

När jag skulle formulera en teststrategi för livscykelhantering utgick jag från checklistan, men jag har försökt tänka på vad som faktiskt går att testa – och kanske viktigare, vad som inte går att testa. Här är de områden där jag anser att vi kan ställa tydliga krav och verifiera dem:

Område	Vad vi testar	Hur vi verifierar
Modulär arkitektur	Går det att byta ut en komponent utan att påverka andra?	Arkitekturgranskning, dokumentation av gränssnitt
Rollback-mekanismer	Kan vi återställa en misslyckad uppdatering?	Simulera fel i testmiljö, mät tid till återställning
API-kompatibilitet	Fungerar integrationer mot flera versioner av journalsystem?	Testautomatisering som körs mot olika versioner
Dokumentation	Finns det aktuell arkitekturdokumentation och ADR:ar?	Granskning vid varje release
Beroendeövervakning	Upptäcker vi föråldrade eller sårbara bibliotek?	Automatiserade verktyg som Snyk eller Dependabot

Det vi inte kan testa – och varför

En insikt jag fått under arbetet är att teststrategi också handlar om att vara ödmjuk inför det man inte kan kontrollera. För 1177.se finns flera områden som lämnas otestade, och det är viktigt att vara medveten om det:

Område som inte kan testas	Varför?	Vad gör vi i stället?
Alla regioners journalsystem samtidigt	21 regioner med olika system och versioner. Att testa alla kombinationer är praktiskt omöjligt.	Övervakning i produktion som upptäcker avvikelser direkt, tydliga felmeddelanden till användare, snabb Rollback.
Fullskalig belastning	Testmiljön kan inte matcha produktionsmiljöns kapacitet och användarbeteende.	Belastningstester i så realistisk miljö som möjligt, men med insikt att första testet i verklig skala sker i produktion.
Långsiktig teknisk skuld	Går inte att "testa" framtida underhållsproblem. Det är en konsekvens av beslut över tid.	Avsätta tid i varje sprint för underhåll, dokumentera designbeslut så att framtida utvecklare förstår varför saker ser ut som de gör.
Användarbeteende över tid	Användare lär sig systemet, hittar genvägar, utvecklar förväntningar. Detta kan inte testas i förväg.	Användarundersökningar, analys av användardata, kontinuerlig uppföljning av användbarhet.

Organisatorisk reflektion – vad krävs av organisationen?

Jag har insett att livscykelhantering egentligen inte är en teknisk fråga – det är en organisationsfråga. Oavsett hur bra teststrategi vi har, kommer den att misslyckas om organisationen inte är byggd för långsiktighet. För 1177.se är utmaningen att 21 regioner gemensamt finansierar systemet. Budgeten är föremål för årliga politiska beslut, vilket gör det svårt att få till en flerårig rambudget trots att systemet behöver det. Hur 1177.se finansieras påverkar direkt möjligheten att arbeta med livscykelhantering:

Budgetmodell	Effekt på livscykelhantering
Årlig budget	Främjar kortsiktighet. Medel som inte används under året kan försvinna – incitament att spendera på nyutveckling snarare än underhåll.
Flerårig rambudget	Möjliggör långsiktig planering. Teknisk skuld kan minskas över tid utan att konkurrera med akuta behov varje år.

Processer – ändringshantering som fungerar

En vanlig utmaning i stora organisationer är att ändringshantering blir så tungrodd att man slutar följa den – eller att alla ändringar behandlas lika oavsett risk.
För 1177.se skulle en effektiv process kunna se ut så här:

Ändringstyp	Risk	Process
Säkerhetspatch	Hög	Express-godkännande, automatisk deployment, övervakning
Ny funktion	Medel	Standard change – testning, dokumentation, godkännande
Uppgradering i region	Hög	Samordnad process med regionens IT-avdelning, test i förväg om möjligt
Större arkitekturförändring	Kritisk	Architecture Decision Record (ADR), pilot i en region först

vem äger egentligen 1177.se över tid? Systemet utvecklas till stor del av konsulter med korta avtal, ofta 1–2 år. Det skapar en stor utmaning:

- **Kunskap försvinner** när konsulter byts ut
- **Inget långsiktigt ägarskap** – vem ansvarar för teknisk skuld över tid?
- **Arkitekturdesign** görs ofta av konsulter som inte finns kvar när systemet ska underhållas

Min slutsats är att det måste finnas krav i avtalen på dokumentation och kunskapsöverföring. Varje konsult ska dokumentera sina designbeslut (ADR) och nyckelkonsulter måste överlappa med efterträdare. Utan det riskerar vi att systemet blir en samling av korta lösningar som ingen längre förstår. Så för att 1177.se ska en bra hantering av sin livscykel så kan det vara bra att tänka på följande punkter.

1. **Flermålsbudget** som avsätter medel för både nyutveckling och underhåll
2. **Effektiv ändringshantering** som anpassas efter risknivå
3. **Långsiktiga avtal** med incitament för dokumentation och kunskapsöverföring
4. **Tydligt ägarskap** – någon har det långsiktiga ansvaret för systemets tekniska hälsa

5. **Kultur** där underhåll värderas lika högt som nyutveckling

Utan dessa organisatoriska förutsättningar spelar det ingen roll hur bra teststrategi vi har – systemet kommer ändå att urholkas över tid.

Avslutande reflektion

När jag började det här arbetet trodde jag att livscykelhantering handlade om versionshantering och testautomation. Nu har jag insett att det handlar om något mycket större. För 1177.se – ett system som ska leva i decennier, som påverkar människors hälsa och som binds samman av 21 regioner med olika förutsättningar – handlar livscykelhantering i grunden om **långsiktigt ansvarstagande**.

Det handlar om att våga prioritera underhåll framför nya funktioner. Det handlar om att skapa avtal och budgetar som möjliggör långsiktighet. Det handlar om att bygga en organisation som kan bära ansvaret för ett system, som är viktigt för regionernas invånare.

Källförteckning

- Olausson, A. M. (2025). *Kvalitetstårtan – En handbok för hållbara system*. andersmolausson.se
- E-hälsomyndigheten. *Nationella riktlinjer för e-hälsa*.
- Patientdatalagen (SFS 2008:355)
- GDPR (EU 2016/679)
- 1177.se